Lab 1: Developing AI Applications with Ollama API

ISA104 AI Application Design

This lab session is designed to be interactive, practical, and educational, giving you a foundational understanding of how AI models can be integrated into real-world applications.

Objectives

1. Understand and use the Ollama API to interact with a small-scale LLM.
2. Design and implement a simple calculator application that can interpret and execute arithmetic operations using natural language input.
3. Integrate an LLM-based interpreter into a basic Python program to demonstrate how AI can be used to understand and process user queries.

Step-by-Step Guide to Completing the Lab

1. **Setting Up the Development Environment**: To begin, you will need to install, configure, and start the Virtual Machine (VM) that has been prepared for this lab. This VM includes all the necessary software components required to develop AI applications using Ollama.

2. **High-Level Architecture Design**: The next step involves sketching out a basic idea of how your software application will function and what its key components are. In software engineering, this conceptual stage is referred to as a high-level architecture design.

3. **Implementation**: For this lab, we'll be focusing on a streamlined version of the software development lifecycle (SDLC). While a full SDLC would typically involve phases such as Planning, Requirement Analysis, Design, Implementation, Testing, Deployment, and Maintenance, our process will focus primarily on Design and Implementation. In this step, you will write Python code to bring your application idea to life.

4. **Testing and Evaluation**: The final step in this lab is to test your application, identify any areas for improvement, and potentially refine it based on your own insights.

Download Lab Image:

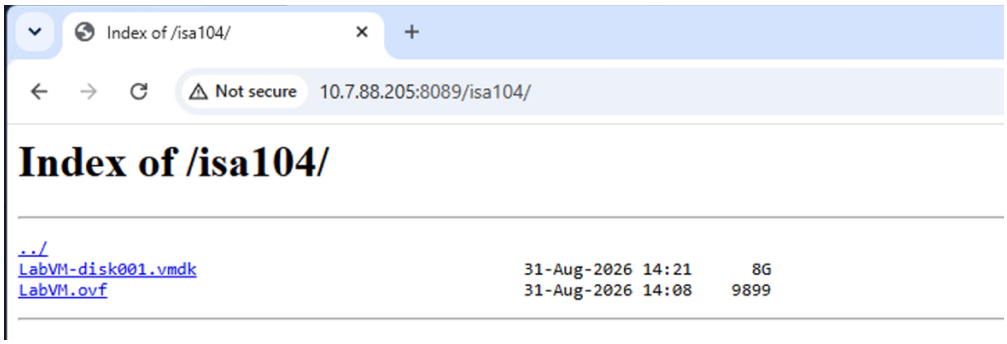


Figure 0. Open your web browser and type the following address to download the lab images (two files). Address: **http://10.7.88.205:8089/isa104/**

Step 1: Setting Up the Development Environment

The pre-configured Virtual Machine (VM) has been equipped with all necessary software components required for this lab, including Ollama APIs and libraries, a Python environment, a code editor (Microsoft VS Code), and a large-language model (Gemma3:270M). To leverage this development environment, please follow the steps outlined below to install, configure, and start your development environment.

Knowledge: A **Virtual Machine (VM)** is a software emulation of a physical computer or hardware platform. It runs an operating system (OS) and applications within a virtual environment, allowing multiple instances to coexist on the same physical machine.

*Alternatively, if you prefer to set up a development environment on your personal computer, you can proceed to Section **[Extra] Setting Up the Development Environment on Your Personal Computer** for instructions on how to replicate the necessary components.*

1. On your Windows machine, click on the Windows icon in the taskbar and type "VirtualBox" in the search bar to locate the Oracle VirtualBox application. Once found, select the VirtualBox icon and click on it to launch the application (Figure 1).

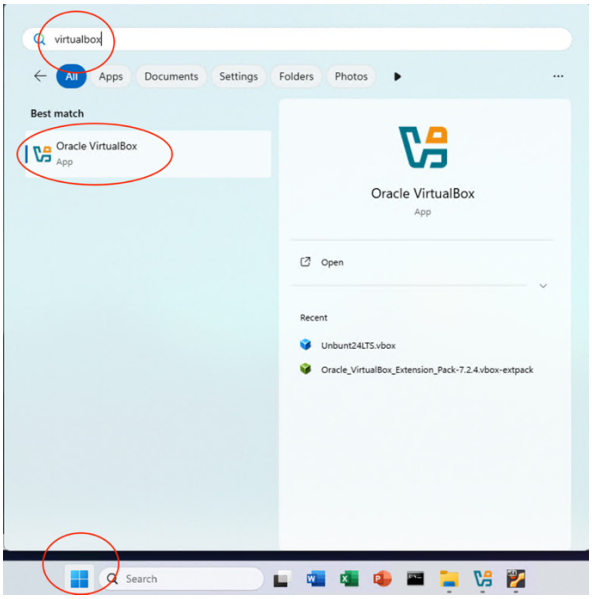


Figure 1. Start the Oracle VirtualBox application.

2. In the popup window, click on the “Import” to import the pre-built VM, as show in Figure 2.

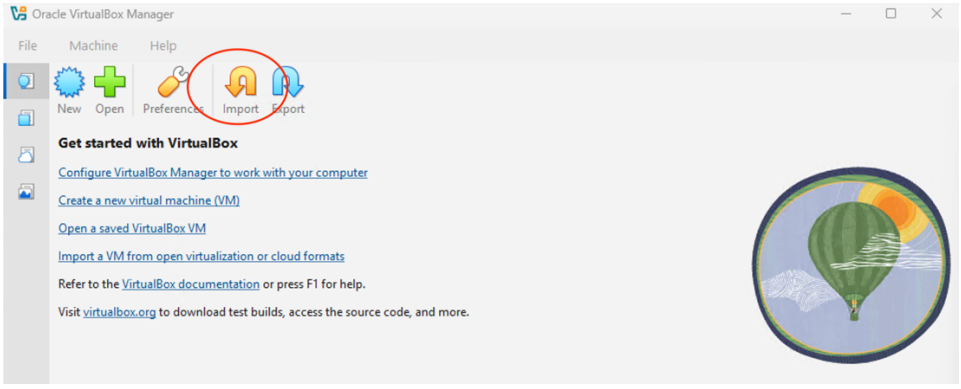


Figure 2. Import a pre-built Virtual Machine

3. Locate the VM image in **.ovf** format and complete the process (Figures 3, 4 & 5).

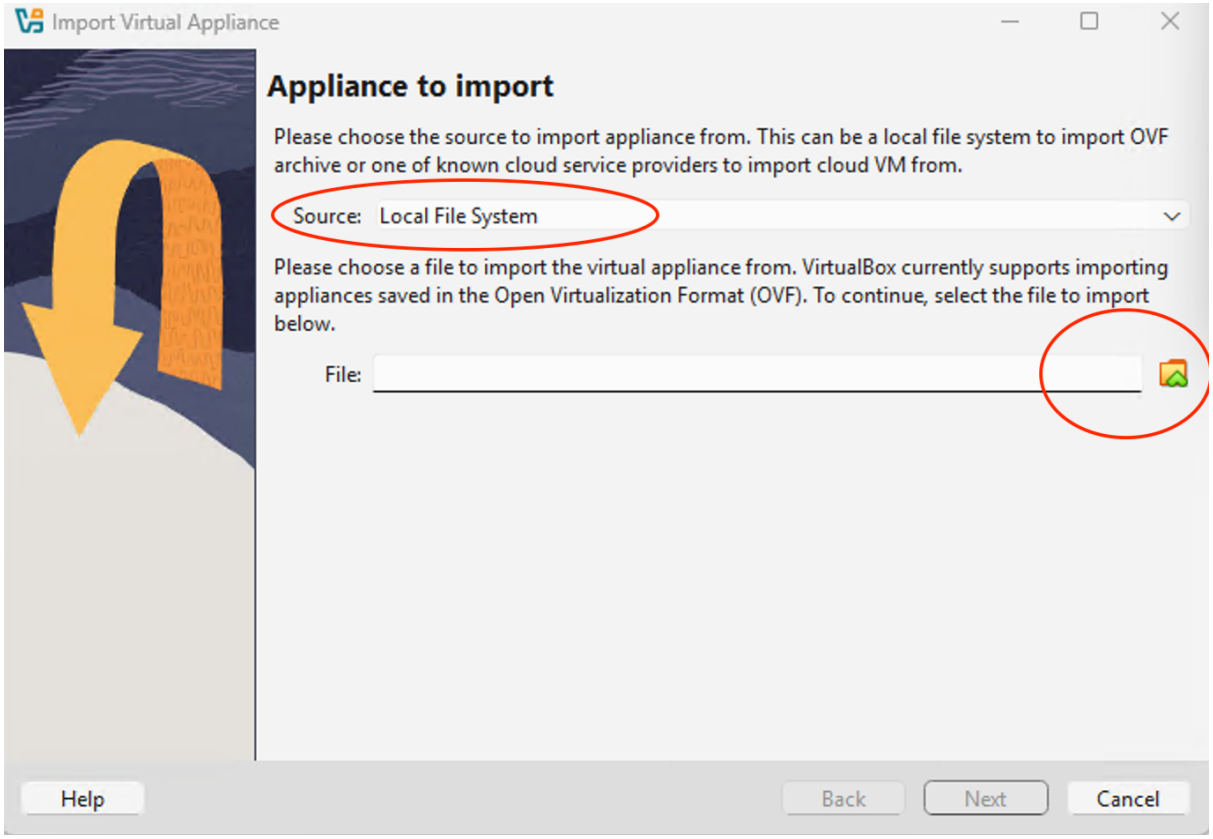


Figure 3. Find your VM image from local file system.

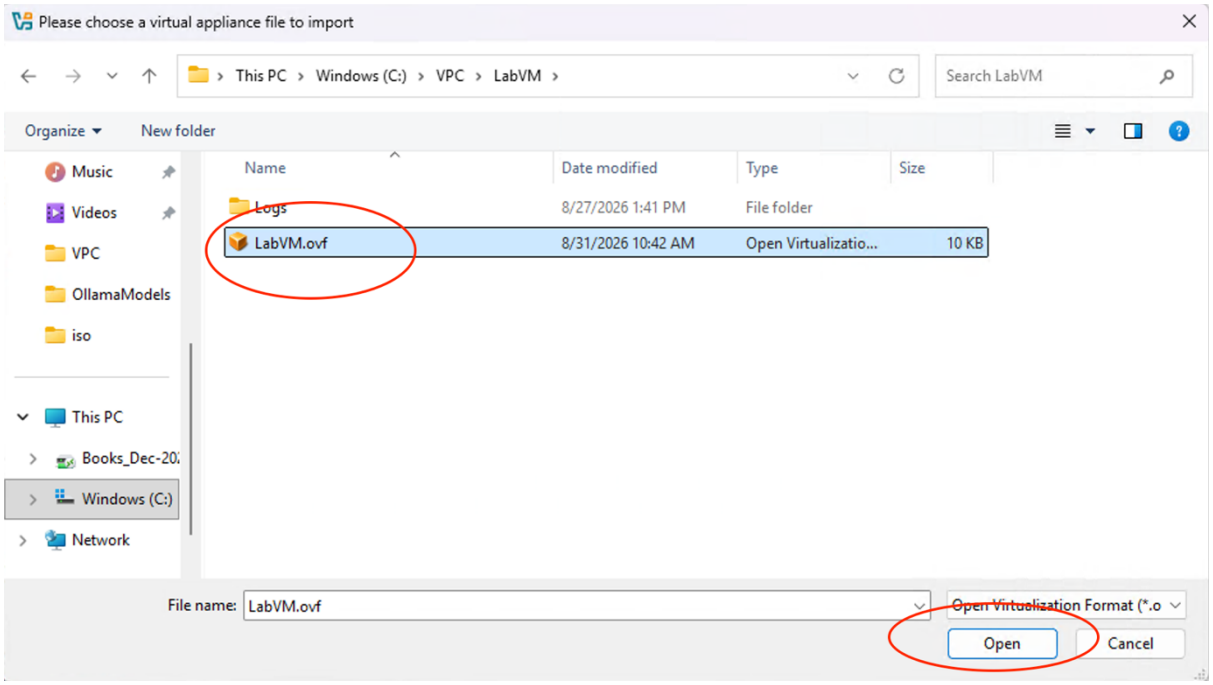


Figure 4. Find the .ovf file (the exact path of the file may not be exactly the same as shown in the screenshot. Important: the files are usually stored in your home folder: **C:\User\<your name>**

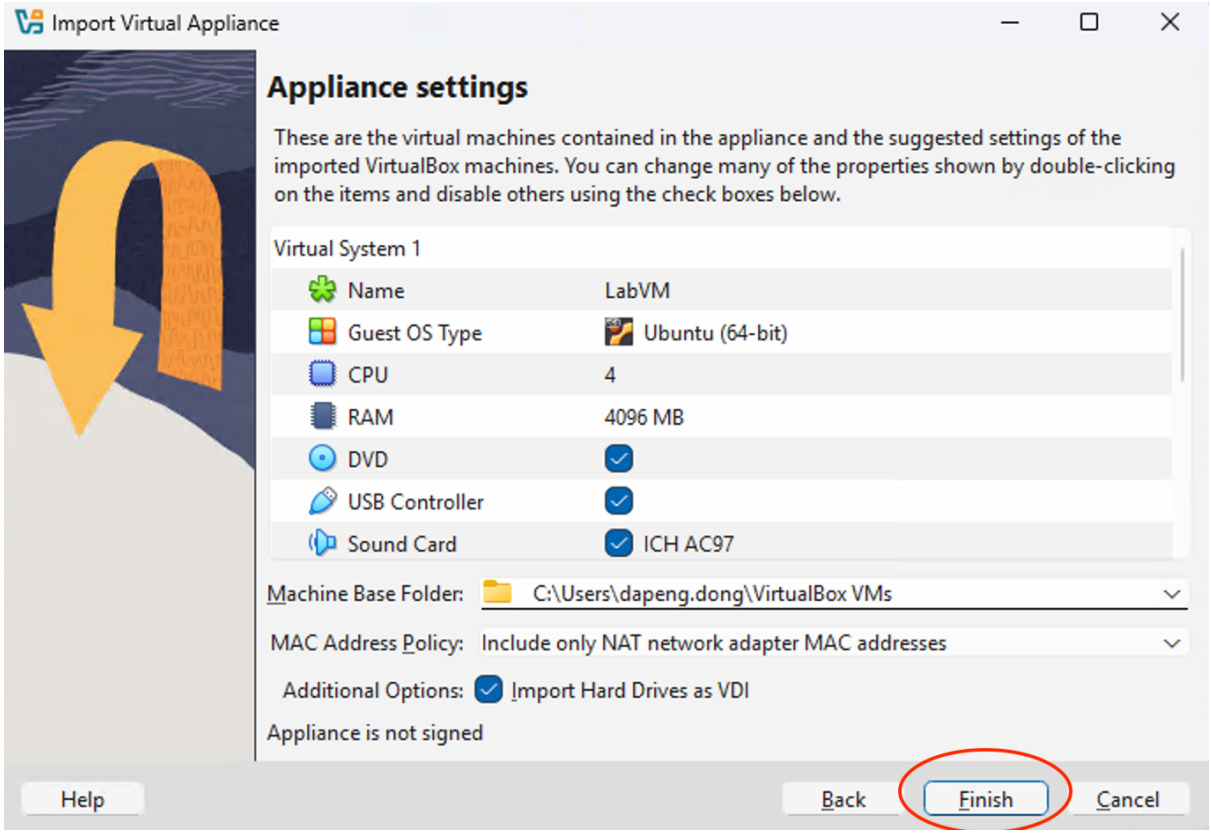


Figure 5. Accept all default configurations and click on "Finish". This importing process may take a few minutes as the image file is large (~8.7GB).

4. Strat the VM by clicking on "Start" (Figure 6).

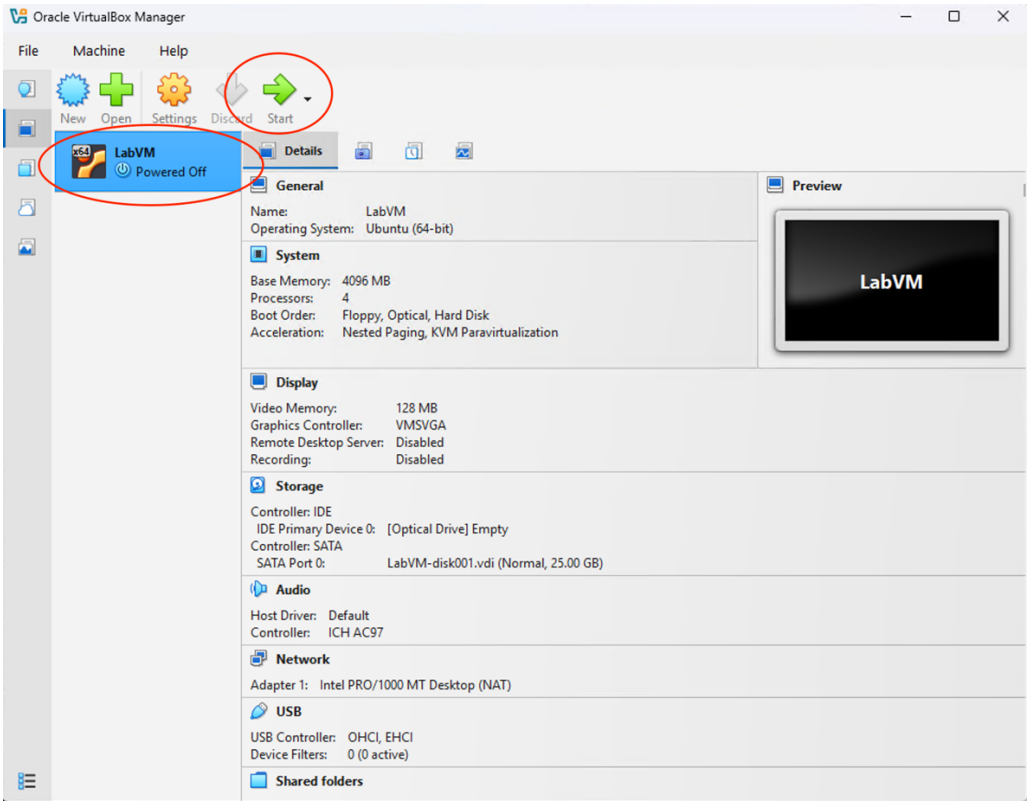


Figure 6. Select the newly imported VM and click on "Start".

5. Login in to your development (Figures 7 & 8).

Username: **xjtlu**
Password: **xjtlu**

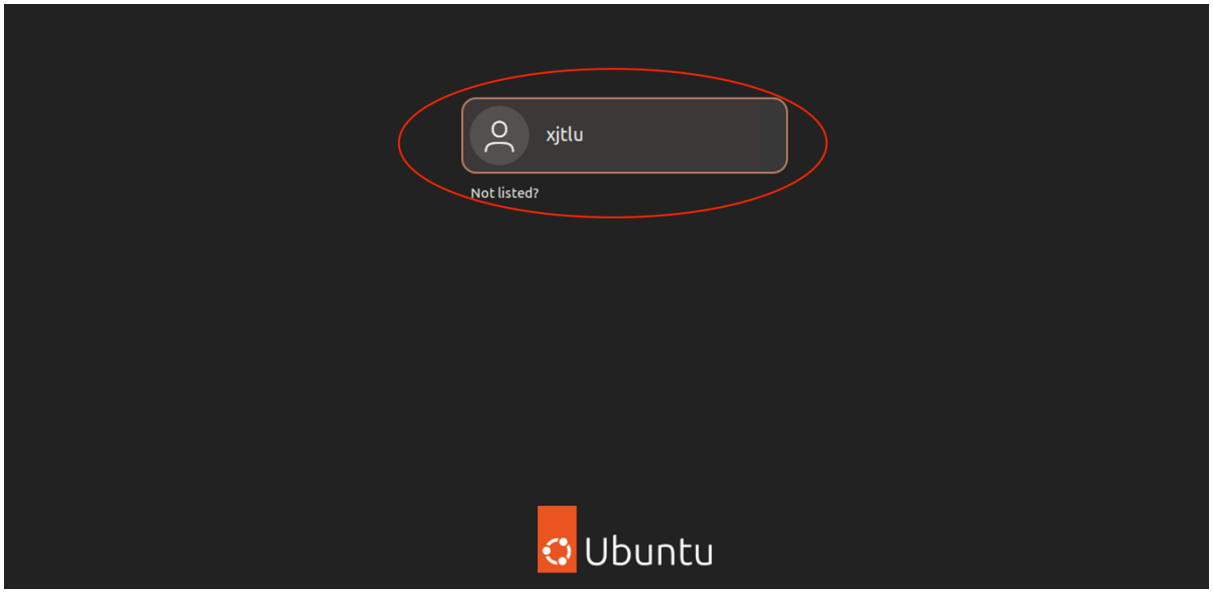


Figure 7. Click on the user "xjtlu"

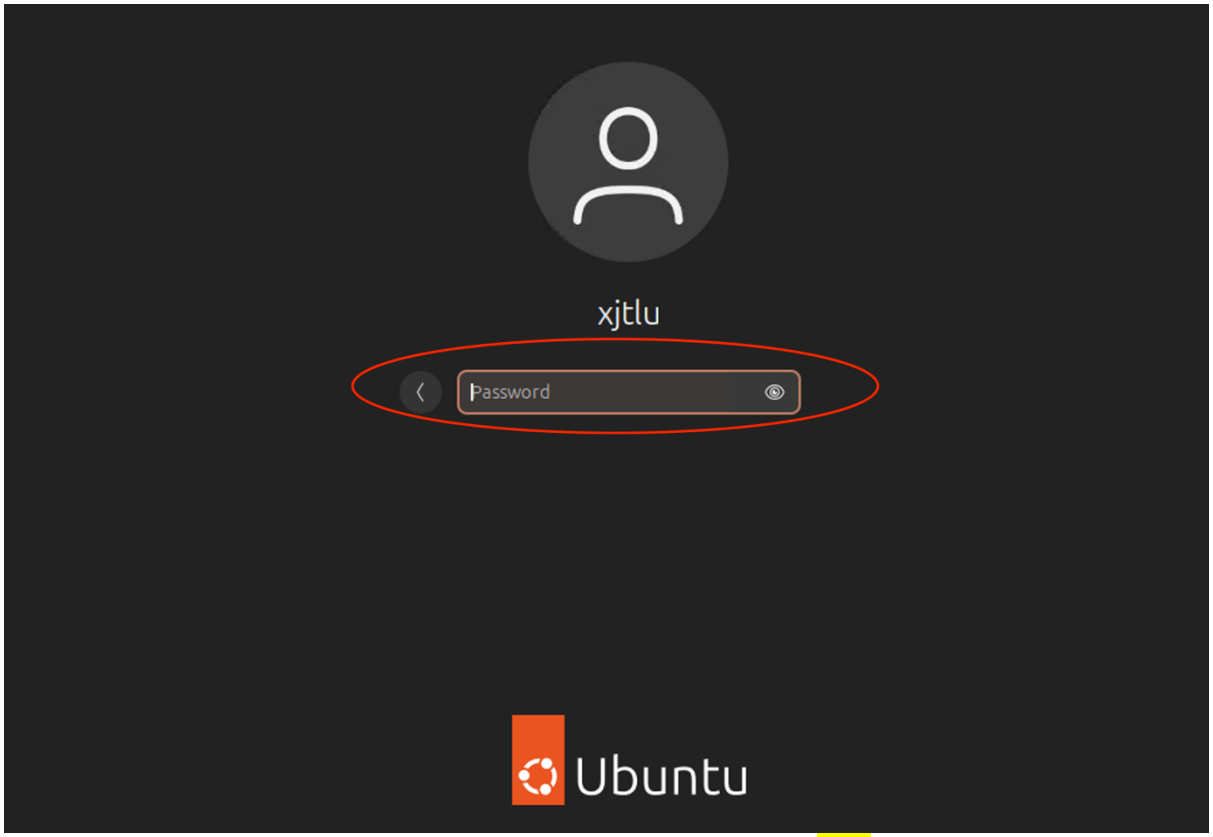


Figure 8. Type in the default password: xjtlu

6. Start the Microsoft VS Code and you are now ready for coding (Figures 9 & 10).

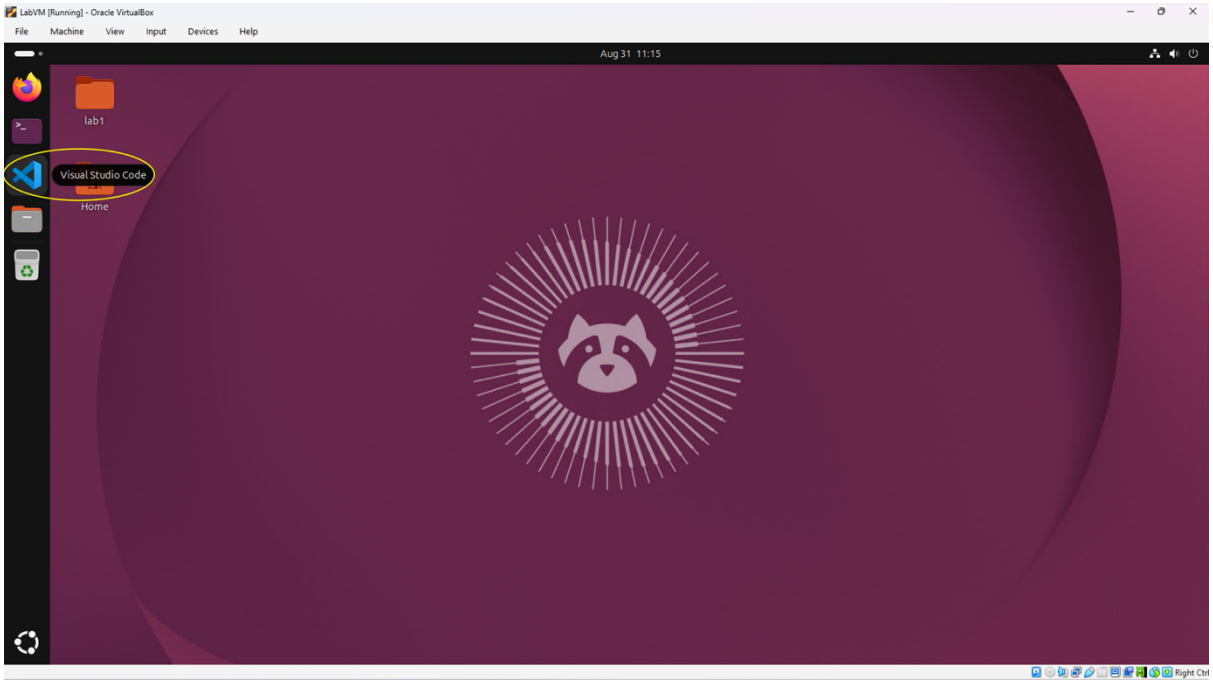


Figure 9. Start VS Code. The app icon is located on the left menu bar.

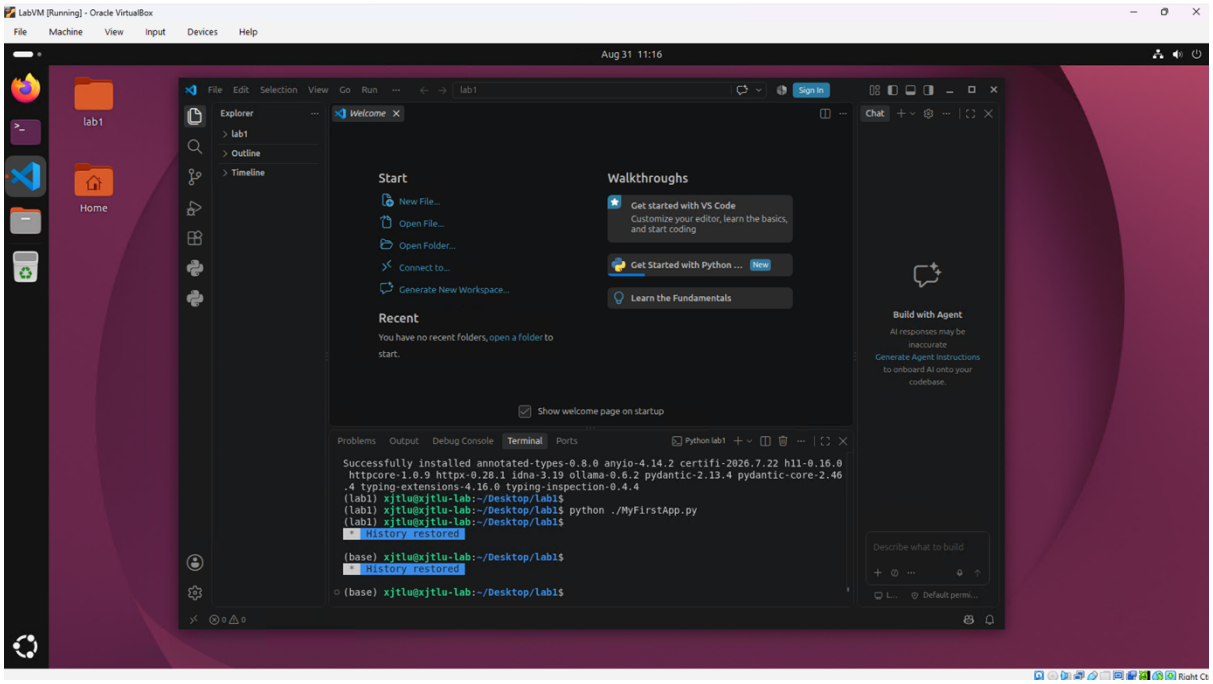


Figure 10. Using the preconfigured environment for Python coding.

[Extra] Setting Up the Development on Your Personal Computer

When developing software systems, having access to a suite of tools and libraries is essential. For this exercise, we will be utilizing Microsoft Visual Studio Code, Ollama, and Python version 3.12.x as core components.

If you choose to complete the exercise on your personal laptop, please follow these steps to configure your development environment:

1. **Install Visual Studio Code**
   Visual Studio Code (VS Code) is a lightweight, open-source code editor developed by Microsoft. It is designed for developers, data scientists, and other technical professionals to write, debug, and manage code efficiently.

   You can download VS Code from the official website (https://code.visualstudio.com/download). After installation, you should be able see a similar window, as shown in Figure 11.

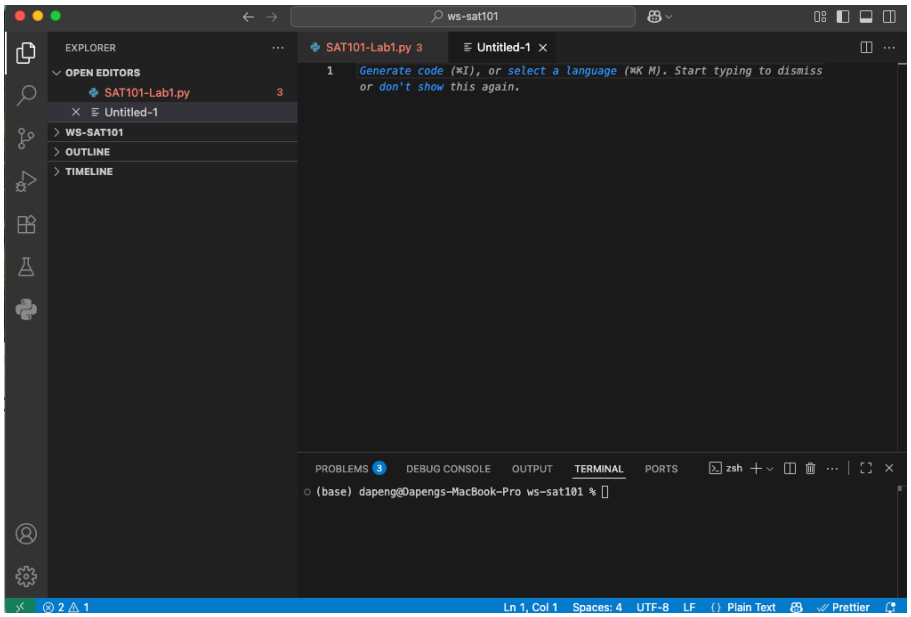


Figure 11. Visual Studio Code (VS Code) Interface

2. **Install Ollama**
   Ollama is a powerful tool for developers and researchers who want to experiment with LLMs locally. Its simplicity and open-source nature make it a suitable choice for integrating AI into projects without relying on cloud platforms. You can download it from its official website at: https://ollama.com.



After installing the software, you need to start a consol terminal from your operating system, and issue the following command to install the Gemma3 270m model to your computer:

```
ollama pull gemma3:270m
```

3. **Install Python 3**

Python is a high-level, interpreted programming language that is widely used for various purposes such as AI, data science, web development, and scientific computing.



Firstly, we need to download a Python installation package from its official website (https://www.python.org). Ensure you download a suitable package for your operating system (e.g., macOS, Linux, or Windows). The installation process is straightforward; simply follow the installation steps step-by-step using the default values.

After install python, you should start a consol terminal from your operating system and install a Python library for Ollama by issuing following command:

```
pip install ollama
```

Knowledge: **Software Libraries** are collections of pre-written code, functions, classes, or methods that developers can use to perform common tasks in software development. They act as reusable components that simplify programming by providing ready-made solutions for specific operations, reducing the need to write code from scratch.

Step 2: High-Leve Architecture Design

The software system is an AI-powered calculator that enables users to interact with a large-language model (LLM) through a graphical user interface (GUI). The system is composed of three main components (illustrated in Figure 12):

1. Python-based User Interface (UI)
2. Ollama API (Acting as the Middleware)
3. Open-Source LLM Backend (Gemma3, 270M Parameters Model by Google DeepMind)

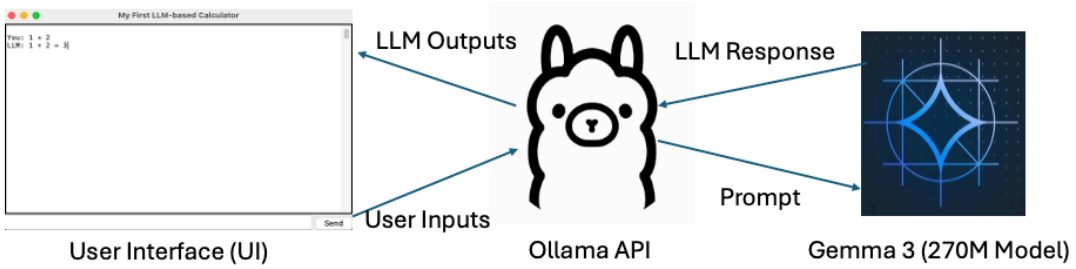


Figure 12. High-level Architecture

***Knowledge*: Software architecture** refers to the high-level structure of a software system. It defines how the system is organized into components, how these components interact with each other, and how they are connected to fulfil the system's functional and non-functional requirements.

System Workflow

1. User Input Collection via the UI
   The Python-based UI is the front-end component that allows users to interact with the system. It provides a simple and intuitive interface for users to input their queries. The UI is built using a Python GUI framework: *Tkinter*.

2. Input Transmission to Ollama API
   Once the user submits input via the UI, i.e., a "send" button, the input is passed to the Ollama API. Ollama acts as the middleware or bridge between the UI and the LLM backend. It is responsible for accepting the user input and formatting it into a structured prompt that the LLM can understand.

3. Processing by the LLM Backend (Gemma3 270M)
   The formatted prompt is then sent to the LLM backend, which is a Gemma3 model with 270 million parameters, hosted locally on your computer and managed through Ollama.

   The Gemma3 model, developed by Google DeepMind, is an open-source, lightweight LLM optimized for performance on local hardware. The model interprets the user's input and generates a contextually relevant response based on its training data and the prompt instructions. The response is returned to the Ollama API.

4. Response Transmission Back to the UI
   The Ollama API receives the response from the LLM and then sends the response back to the Python-based UI. The UI presents it to the user in a text box.

Step 3: Implementation

The code is divided into several parts, which are illustrated in Figure 13. First, let use the pre-configured project workspace called "**lab1**", as shown in Figure 14. Within VS Code, under the "lab1" folder, create our Python file with a "*.py*" extension, indicating that it is a Python file. For example, you may call it *MyFirstApp.py* (Figure 15).

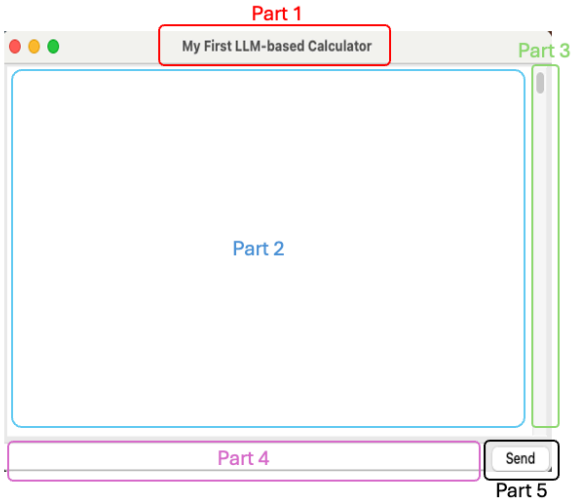


Figure 13. The GUI components are shown in relation to their corresponding code segments.

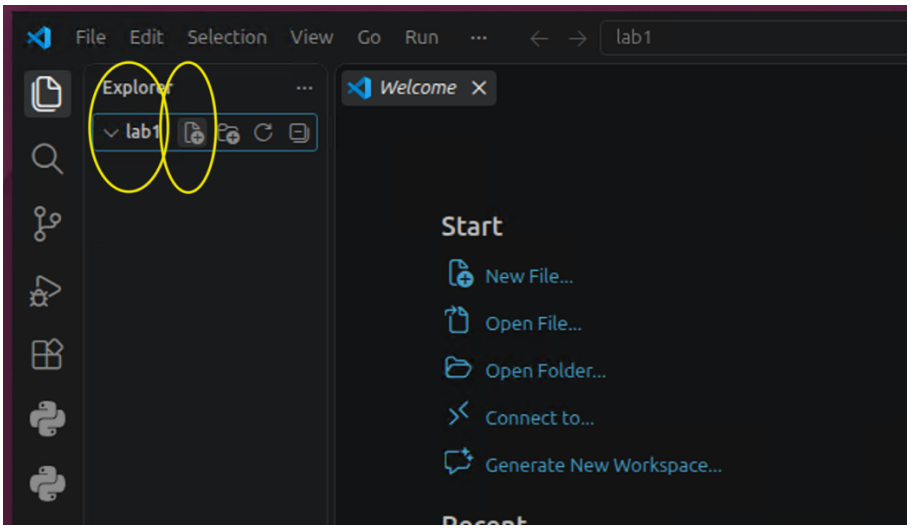


Figure 14. Find the preconfigured workspace ("lab1") and create a new file.

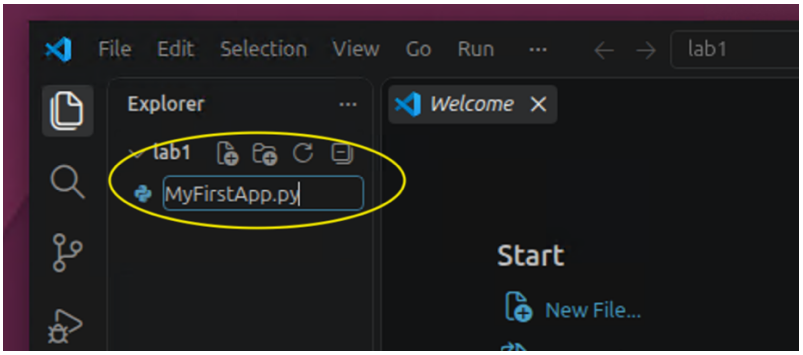


Figure 15. Name the new file "MyFirstApp.py"

```
# Making the Python GUI library (tkinter) available to your application
from tkinter import *

# Making the Ollama Application Programming Interfaces (APIs) available to your application
from ollama import chat
```

```
# Part 1
# Creating an empty GUI window
root = Tk( )

# Setting the title of your window
root.title("My First LLM-based Calculator")
```

```
# Part 2
# Creating a text box for displaying user input and the responses from LLM.
txt = Text(root, bg="white", fg="black")
txt.grid(row=1, column=0, columnspan=2)
```

```
# Part 3
# Creating a scrollbar on the right side of the window
scrollbar = Scrollbar(txt)
scrollbar.place(relheight=1, relx=0.975)
```

```
# Part 4
# Creating an input area for users
e = Entry(root, bg="white", fg="black", width=55)
e.grid(row=2, column=0)
```

```
# Part Ollama
# Creating an action for the "Send" button and connect the GUI to Ollama
def send( ):
    send = e.get( )
    txt.insert(END, "\n" + "You: " + send)

    response = chat(
        model='gemma3:270m',
        messages=[{'role': 'user', 'content': send}]
    )

    txt.insert(END, "\n" + "LLM: " + response['message']['content'])
    e.delete(0, END)
```

```
# Part 5
# Creating a "Send" button. When the button is clicked, the user input will be sent to LLM via Ollama
send = Button(root, text="Send", command=send).grid(row=2, column=1)
```

```
# The program will run forever until you close the window.
root.mainloop( )
```

Knowledge: In Python, **the '#' symbol** represents a *comment*. When you prefix a line with '#', everything on that line after the '#' is ignored by the interpreter.

Step 4: Testing and Evaluation

After coding your calculator, ensure that you have saved the file. Then you need to start the Python environment that contains the Ollama API and libraries. These have been pre-installed for you. See Figure 16 for details.

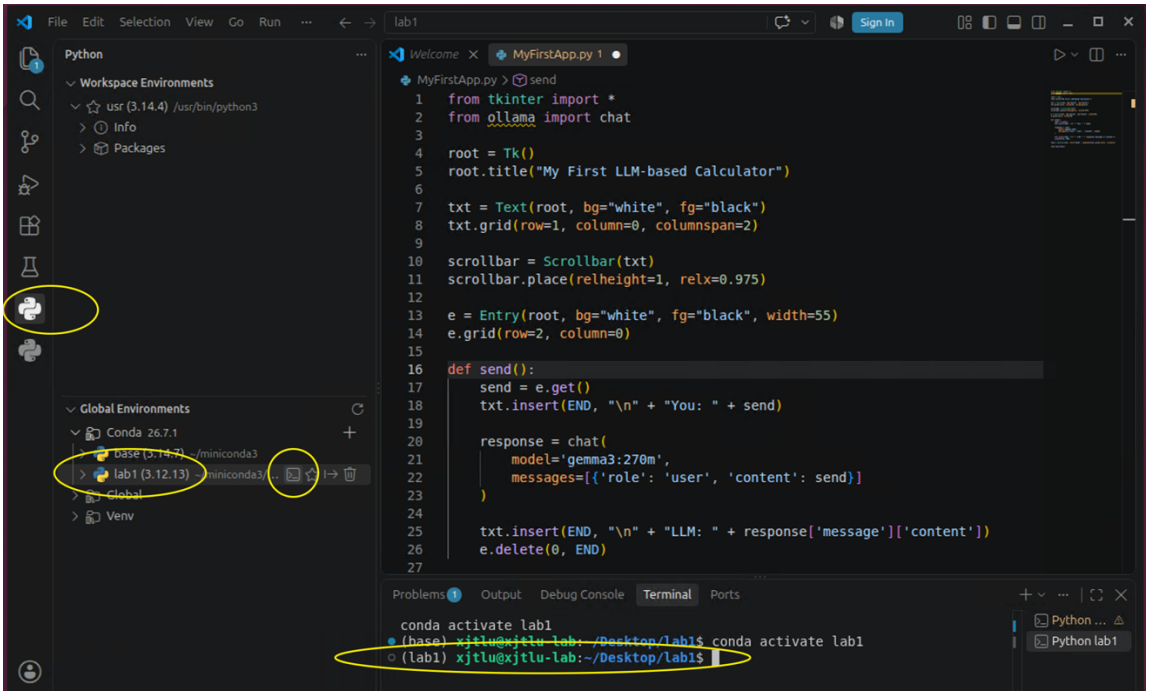


Figure 16. Start the Python environment. Locate the Python manager located on the left side of the window. Under the "Conda" option, find "lab1", then click on the terminal icon. You should see a new Python environment showing at the bottom of the screen similar to "(labl1) xjtlu@xjtlu-lab:~/Deskotp/lab1$".

You can run you application using the following command, and you should then be able to see a GUI window that resembles the one shown in Figure 17.

```
python MyFirstApp.py
```

```
(lab1) xjtlu@xjtlu-lab:~/Desktop/lab1$ python MyFirstApp.py
```

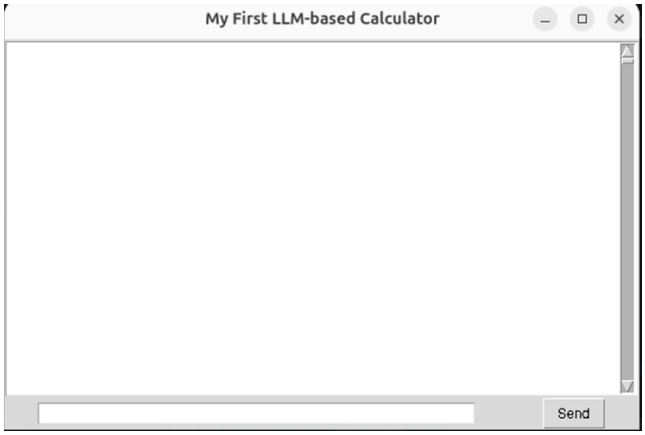


Figure 17. GUI Interface of your calculator application

Test your program with some examples:

```
1 + 2 + 3 * 7 =

1 + (3 + 7) * 2 – 1 =
```

Which test case gives you the correct answer and which one is not? How do you improve it?

Hint: 1 + 2 * (3 + 7) – 1 =

Highlight: AI-based applications must prioritize key factors such as reliability, stability, scalability, and accuracy to ensure effective and sustainable performance.

Summary

Now you have completed your first AI-powered application. Its design is relatively simple and it is much simpler compared to a conventional method for programming a calculator, which requires all arithmetic operators to be hard-coded and involves many efforts in dealing with special cases and exceptions.

On the other hand, a correctly implemented calculator application using conventional methods guarantees the correctness of the result it produces, but an AI-powered calculator may not always provide the right answer. Therefore, many aspects need to be taken into account when designing an AI-powered application.

Can you list any other aspects that a designer or developer needs to pay attention to when designing AI-powered applications?

Hints:

- Scalability: Can your AI-powered application handle a large number of users or data inputs without degrading performance?
- Transparency: Is it clear how your AI model makes decisions and produces results? Can users understand and trust the output?
- Privacy: Does your application collect and process sensitive user data? How do you protect user privacy and ensure compliance with regulations?
- Security: Are you vulnerable to attacks or data breaches? Do you implement robust security measures to protect against unauthorized access?
- Ethical considerations: Is your AI model biased or discriminatory in any way? Have you considered the potential social implications of your application?
- Stability: Does your application function reliably and consistently, even under unexpected inputs or edge cases?
- Reliability: Can users rely on your application to produce accurate results, or is it prone to errors or crashes?
- Controllability: Is it possible for users to influence the behavior of your AI model or adjust its parameters to suit their needs?
- Accuracy: How does your AI-powered application compare in terms of accuracy and precision to traditional methods or human performance?
- Speed: Can your application process inputs quickly enough to meet user expectations, especially when dealing with large datasets?